

JAVASCRIPT

Session 1 Notes

Variables • Data Types • User Input • Basic Operations

Beginner-Friendly Study Guide

Concept Explanation + Examples + Revision + Practice

Core topics: `console.log()`, variables, declaration, initialization, `var/let/const`, redeclaration, reassignment, primitive data types, arrays, objects, `prompt()`, `Number()`, and `typeof`.

Table of Contents

Use this page as a navigation map for the session.

Section	Topic
1	Introduction to JavaScript
2	First JavaScript Program
3-6	Variables, Declaration, Initialization
7-9	var, let, const, Redefinition, Reassignment
10-17	Data Types: Primitive Values
18-20	Arrays and Objects
21-24	prompt(), Number(), Addition, typeof
25-26	Common Mistakes and Comparison Tables
27	Quick Revision Sheet
28	Practice Questions
29	Mini Practical Task
30	Practice Answer Guide
31	Session Summary
Appendix	Cleaned Session 1 Source Code

Navigation tip

The PDF also contains clickable PDF outline/bookmark entries in compatible readers.

1. Introduction to JavaScript

JavaScript is a programming language widely used to add behavior and interactivity to web pages and web applications. In a browser, JavaScript can respond to user actions, work with data, and change page content. It can also run outside the browser in environments such as Node.js.

Session goal

By the end of this session, you should be able to declare variables, understand their basic rules, identify common data types, take simple browser input, convert that input to numbers, and perform basic arithmetic.

2. First JavaScript Program

The first example is a simple console output statement.

```
console.log("Hello");
```

What happens? `console.log()` prints a value to the JavaScript console. In a browser, the console can be opened through Developer Tools. The text "Hello" is a string value.

Tip

`console.log()` is one of the simplest ways to check what a variable contains while you are learning and debugging.

Expected output:

```
Hello
```

3. Variables in JavaScript

A variable is a named binding that lets your program refer to a value. JavaScript provides three main variable declaration keywords: `var`, `let`, and `const`.

```
var a;  
let b;  
const c = 10;
```

These keywords do not behave identically. The main differences in this session are whether a variable may be initialized later, redeclared, or reassigned.

4. Declaration

Declaration means introducing a variable name to JavaScript. With `var` and `let`, you can declare first and provide a value later.

```
var a;  
let b;
```

- **var a;** declares a.
- **let b;** declares b.
- At this point, neither statement explicitly provides an initial value.

A `const` variable is different: it must have an initializer in the declaration.

```
// Invalid
const c;

// Valid
const c = 10;
```

Important

The accurate rule is: a const declaration must include an initializer. Avoid the incorrect phrase "constant variable cannot be declared."

5. Initialization

Initialization means giving a variable an initial value. For `var` and `let`, that value can be assigned after the declaration.

```
var a;
let b;

a = 10;
b = 20;
```

Think of the sequence as:

```
Declaration  -> Initialization
let b;        -> b = 20;
```

6. Declaration + Initialization

You can also declare a variable and assign its initial value in the same statement.

```
var x = 70;
let y = 20;
const z = 43;
```

For `let y = 20;`: `let` is the keyword, `y` is the variable name, and `20` is the initial value.

7. var, let and const

The following table captures the practical differences needed for this session.

Feature	var	let	const
Declaration without initialization	Yes	Yes	No
Redeclaration in the same scope	Yes	No	No
Reassignment	Yes	Yes	No
Must initialize immediately	No	No	Yes

8. Re-declaration

Redeclaration means declaring the same variable name again in the same scope.

```
var x = 70;
var x = 40;

console.log(x); // 40
```

With `var`, redeclaration in the same scope is allowed. With `let` and `const`, redeclaration in the same scope is not allowed.

```
let y = 20;
// let y = 40; // SyntaxError

const z = 43;
// const z = 50; // SyntaxError
```

9. Re-assignment

Reassignment means changing the value of an already declared variable.

```
var x = 40;
let y = 20;
const z = 43;

x = 66;
y = 44;

console.log(x); // 66
console.log(y); // 44

const z = 43;
z = 50; // TypeError
```

Concept	Meaning	Example
Redeclaration	Declare the same variable name again	<code>var x = 70;</code> then <code>var x = 40;</code>
Reassignment	Change the value of an existing variable	<code>let y = 20;</code> then <code>y = 44;</code>

Remember Redeclaration and reassignment are different operations. A variable can be reassigned without being redeclared.

10. Data Types in JavaScript

Data types describe the kind of value a variable refers to. At this level, it is useful to separate the values in your session into primitive values and object/reference types.

JavaScript Data Types

-
- Primitive
 - ■■■ String
 - ■■■ Number
 - ■■■ Boolean
 - ■■■ Undefined
 - ■■■ Null
 - ■■■ BigInt
-
- Object / Reference Types
 - Array
 - Object

11. Primitive Data Types

Primitive values represent single, basic values and are not objects. The six primitive types covered in this session are string, number, boolean, undefined, null, and bigint.

Accuracy note

It is better to avoid a simplistic claim that primitives are always "stored directly" while objects are always "stored by reference." For beginner learning, focus on the fact that primitives and objects behave differently and that arrays/objects are commonly treated as reference types.

12. String

A string represents text. String literals can be written with single or double quotes.

```
let str = "Hello";
let name = "Roshan";
let city = 'Mumbai';
```

13. Number

JavaScript uses the number type for both integers and floating-point values.

```
let num = 56;
let num1 = 12.43;

// 56      -> integer
// 12.43    -> floating-point number
```

Both 56 and 12.43 have the type number.

14. Boolean

A Boolean value has one of two values: true or false.

```
let isReady = true;
isReady = false;
```

Beginner analogy

You can think of true/false as an on/off or condition-satisfied/condition-not-satisfied signal. The actual JavaScript type is Boolean.

15. Undefined

If you declare a variable with `let` or `var` and do not provide a value, the variable normally has the value `undefined`.

```
let value;  
console.log(value); // undefined  
console.log(typeof value); // "undefined"
```

16. Null

`null` is commonly used to represent an intentional absence of a value.

```
let nullVal = null;
```

Value	Beginner interpretation
undefined	No value has been assigned yet / value is absent by default
null	Programmer intentionally represents "no value"

```
typeof null; // "object"
```

This result is a historical quirk of JavaScript. It does not mean that `null` should be taught as an ordinary object.

17. BigInt

`BigInt` is used for integers that need a range or precision beyond what ordinary JavaScript `Number` can safely represent as exact integers.

```
let bigNum = 2342323534523453464575686785745n;  
let largeNumber = 9007199254740991n;
```

The `n` suffix identifies a `BigInt` literal.

18. Non-Primitive / Reference Types

Arrays and objects are object values and are commonly described as reference types. For beginner-level understanding, this means a variable referring to an array or object refers to that object value rather than containing a primitive value like a string or number.

19. Array

An array stores an ordered collection of values. Indexing starts at 0. Duplicate values are allowed.

```
const arr = [1, 2, 3, 4, 5, 6, 6, 7];
```

Index	0	1	2	3	4	5	6	7
Value	1	2	3	4	5	6	6	7

```
console.log(arr[0]); // 1  
console.log(arr[3]); // 4
```

20. Object

An object stores data using key-value pairs. In the example below, `id` and `key` are property names, and 4 and 78 are their values.

```
const obj = {
  id: 4,
  key: 78
};

console.log(obj.id);
console.log(obj.key);
console.log(obj["id"]);
```

The dot notation and bracket notation both let you access an object property.

21. Taking User Input with `prompt()`

In a browser, `prompt()` displays a dialog asking the user for input. The value returned from `prompt` is normally a string.

```
let s = prompt("Enter Num1: ");
let g = prompt("Enter Num2: ");
```

Important

If the user types 10, `prompt()` gives your program the text value "10". It is not automatically a numeric value just because the text contains digits.

22. Converting Input with `Number()`

Use the `Number()` function to convert a numeric string to a JavaScript number.

```
let s = Number(prompt("Enter Num1: "));
let g = Number(prompt("Enter Num2: "));
```

```
"10" -> Number("10") -> 10
```

Why conversion matters:

```
"10" + "20"
// "1020"

Number("10") + Number("20")
// 30
```

The first expression concatenates strings. The second performs numeric addition.

```
User input
  |
  v
prompt()
  |
  v
String
  |
  v
Number()
  |
  v
Number value
  |
  v
Arithmetic
```

23. Add Two Numbers Program

This program combines `prompt()`, `Number()`, `arithmetic`, `console.log()`, and `typeof` into one small exercise.

```
let s = Number(prompt("Enter Num1: "));
let g = Number(prompt("Enter Num2: "));

console.log(
  "Addition:",
  s + g,
  "\nType of Number:",
  typeof (s + g)
);
```

Line-by-line flow:

- **Step 1:** Ask the user for the first value.
- **Step 2:** Convert the returned string to a number.
- **Step 3:** Ask for the second value.
- **Step 4:** Convert it to a number.
- **Step 5:** Calculate `s + g`.
- **Step 6:** Use `typeof` to check the result type.

Example run:

```
Enter Num1: 10
Enter Num2: 20

Addition: 30
Type of Number: number
```

24. typeof Operator

The `typeof` operator tells you the type of a value.

Expression	Typical result
<code>typeof "Hello"</code>	<code>"string"</code>

Expression	Typical result
typeof 56	"number"
typeof true	"boolean"
typeof undefined	"undefined"
typeof 123n	"bigint"
typeof []	"object"
typeof {}	"object"
typeof null	"object" (historical quirk)

25. Common Beginner Mistakes

These are the mistakes most likely to cause confusion when learning the session.

Mistake 1 - Declaring const without an initializer

```
const x; // Error
```

A const declaration must include an initial value.

Mistake 2 - Reassigning a const variable

```
const x = 10;
x = 20; // TypeError
```

const prevents reassignment of that binding.

Mistake 3 - Assuming prompt() returns a number

```
let a = prompt("Enter number");
```

The returned input is normally a string, so numeric input often needs conversion with Number().

Mistake 4 - Forgetting Number()

```
let a = prompt("Enter: ");
let b = prompt("Enter: ");
console.log(a + b);
```

If the user enters 10 and 20, the output can be 1020 because the values are strings.

Mistake 5 - Confusing redeclaration and reassignment

Operation	What changes?	Example
Redeclaration	A declaration is made again	var x = 70; then var x = 40;
Reassignment	The current value is changed	let y = 20; then y = 44;

26. Comparison Tables

Variable keyword	Redeclare?	Reassign?	Initialize immediately?
var	Yes	Yes	No
let	No (same scope)	Yes	No
const	No (same scope)	No	Yes

Data type	Example	Main idea
String	"Hello"	Text
Number	56, 12.43	Numeric values
Boolean	true	true / false values; logical conditions
Undefined	undefined	No assigned value
Null	null	Intentional absence of value
BigInt	123n	Very large integers
Array	[1,2,3]	Ordered collection
Object	{id:4}	Key-value data

27. Quick Revision Sheet

Variables	var -> redeclaration + reassignment allowed let -> reassignment allowed, same-scope redeclaration not allowed const -> reassignment not allowed; initializer required
Primitive types	String - Number - Boolean - Undefined - Null - BigInt
Objects / reference types in this session	Array - Object
Useful functions / operator	prompt() -> receives browser input Number() -> converts values to numbers typeof -> checks a value type

28. Practice Questions

Basic

- 1. Print "Hello JavaScript" using console.log().
- 2. Declare a variable using var.
- 3. Declare a variable using let.
- 4. Declare and initialize a const.
- 5. Create a string variable.

- 6. Create a Boolean variable.

Intermediate

- 7. Take two numbers using `prompt()` and find their sum.
- 8. Take two numbers and find their subtraction.
- 9. Take two numbers and find their multiplication.
- 10. Display the type of a numeric result using `typeof`.
- 11. Create an array and access its third element.
- 12. Create an object and access one of its properties.

Conceptual

- 13. What is the difference between declaration and initialization?
- 14. What is the difference between redeclaration and reassignment?
- 15. Why is `Number(prompt())` commonly used for arithmetic input?

29. Mini Practical Task - Simple Calculator

Build a small browser-based calculator using only the ideas from Session 1.

- Take two numbers using `prompt()`.
- Convert both inputs using `Number()`.
- Calculate addition, subtraction, multiplication, and division.
- Print the results using `console.log()`.
- Use `typeof` to display the type of at least one numeric result.

Practice goal

Do not copy a solution first. Try to build the calculator from the concepts you learned in this session.

30. Practice Answer Guide

Use these only after attempting the exercises yourself.

```
// 1
console.log("Hello JavaScript");

// 2
var age = 21;

// 3
let city = "Mumbai";

// 4
const pi = 3.14;

// 5
let message = "Hello";

// 6
let isReady = true;

// 7
let a = Number(prompt("Enter first number: "));
let b = Number(prompt("Enter second number: "));
console.log(a + b);
```

For exercises 8-12, use the same input/array/object techniques demonstrated earlier. The point is to reproduce the pattern independently rather than memorize one fixed answer.

31. Session Summary

This session introduced the basic building blocks needed to begin writing JavaScript programs.

- `console.log()` prints values to the console.
- Variables can be declared with `var`, `let`, and `const`.
- Declaration introduces a variable; initialization gives it a value.
- Redefinition and reassignment are different operations.
- Primitive types covered: string, number, boolean, undefined, null, and bigint.
- Arrays and objects are object/reference types covered in this session.
- `prompt()` receives browser input, normally as a string.
- `Number()` converts suitable values to numbers.
- `typeof` helps identify the type of a value.

What You Should Be Able to Do Now

- Declare and initialize variables confidently.
- Explain the basic differences among `var`, `let`, and `const`.
- Distinguish redeclaration from reassignment.
- Recognize the six primitive types introduced in the session.
- Create simple arrays and objects.
- Take numeric input using `prompt()` and `Number()`.
- Perform basic arithmetic and inspect result types with `typeof`.

**Session 1
complete**

Next sessions can build on these fundamentals by introducing operators, conditions, loops, functions, and more JavaScript behavior.

Appendix - Cleaned Session 1 Source Code

A cleaned, consistently formatted version of the classroom code used in this session.

```
console.log("Hello");

// Variables
var a;
let b;
const c = 10;

// Initialization
a = 10;
b = 20;

// Declaration + initialization
var x = 70;
let y = 20;
const z = 43;

// Redeclaration (allowed for var)
var x = 40;

// Reassignment (allowed for var and let)
x = 66;
y = 44;

// Primitive data types
let str = "Hello";
let num = 56;
let num1 = 12.43;
let isReady = true;
isReady = false;
let nullVal = null;
let bigNum = 2342323534523453464575686785745n;

// Arrays and objects
const arr = [1, 2, 3, 4, 5, 6, 6, 7];

const obj = {
  id: 4,
  key: 78
};

// Add two numbers using prompt()
let s = Number(prompt("Enter Num1: "));
let g = Number(prompt("Enter Num2: "));

console.log(
  "Addition:",
  s + g,
  "\nType of Number:",
  typeof (s + g)
);
```